

Introduction to Machine Learning for Text Analysis with Python

Day 3 – Wednesday

»Supervised Machine Learning«

Rupert Kiddle
Sjoerd Stolwijk

r.t.kiddle@vu.nl, @rptkiddle
s.b.stolwijk@uu.nl

September 16, 2026

Today

Recap

Evaluation

Supervised Machine Learning for Text Classification

(Traditional) non-SML approaches

Traditional Machine Learning

The implementation

ML versus dictionaries

The explainability of ML

Exercise

Recap

	Methodological approach		
	<i>Counting and Dictionary</i>	<i>Supervised Machine Learning</i>	<i>Unsupervised Machine Learning</i>
Typical research interests and content features	visibility analysis sentiment analysis subjectivity analysis	frames topics gender bias	frames topics
Common statistical procedures	string comparisons counting	support vector machines naive Bayes	principal component analysis cluster analysis latent dirichlet allocation semantic network analysis
			

Boumans and Trilling, 2016

How Humans Learn vs. How Machines Learn

Human Learning

Example counting

(seeing many repetitions)

Pointing out

(explicit correction / feedback)

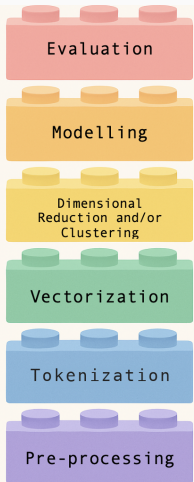
Explaining in other words

(paraphrase / abstraction)

Interpreting context

(discourse / world knowledge)

Lego pipeline



how to assess the output of your model

how to train a model to describe (UML)
or to predict (SML)

how to group features *or* cases

how to turn text → numbers

how to split text into tokens

how to tidy up text data

You have done it before!

You have done it before!

Regression

1. Based on your data, you estimate some regression equation

$$y_i = \alpha + \beta_1 x_{i1} + \dots + \beta_p x_{ip} + \varepsilon_i$$
2. Even if you have some *new unseen data*, you can estimate your expected outcome $\hat{y}$!
3. Example: You estimated a regression equation where y is newspaper reading in days/week:

$$y = -.8 + .4 \times man + .08 \times age$$
4. You could now calculate $\hat{y}$ for a man of 20 years and a woman of 40 years – *even if no such person exists in your dataset*:

$$\hat{y}_{man20} = -.8 + .4 \times 1 + .08 \times 20 = 1.2$$

$$\hat{y}_{woman40} = -.8 + .4 \times 0 + .08 \times 40 = 2.4$$

You have done it before!

Regression

1. Based on your data, you estimate some regression equation

$$y_i = \alpha + \beta_1 x_{i1} + \cdots + \beta_p x_{ip} + \varepsilon_i$$

2. Even if you have some *new unseen data*, you can estimate your expected outcome $\hat{y}$!

3. Example: You estimated a regression equation where y is newspaper reading in days/week:

$$y = -.8 + .4 \times man + .08 \times age$$

4. You could now calculate $\hat{y}$ for a man of 20 years and a woman of 40 years – *even if no such person exists in your dataset*:

$$\hat{y}_{man20} = -.8 + .4 \times 1 + .08 \times 20 = 1.2$$

$$\hat{y}_{woman40} = -.8 + .4 \times 0 + .08 \times 40 = 2.4$$

You have done it before!

Regression

1. Based on your data, you estimate some regression equation
$$y_i = \alpha + \beta_1 x_{i1} + \cdots + \beta_p x_{ip} + \varepsilon_i$$
2. Even if you have some *new unseen data*, you can estimate your expected outcome $\hat{y}$!

3. Example: You estimated a regression equation where y is newspaper reading in days/week:

$$y = -.8 + .4 \times man + .08 \times age$$

4. You could now calculate $\hat{y}$ for a man of 20 years and a woman of 40 years – *even if no such person exists in your dataset*:

$$\hat{y}_{man20} = -.8 + .4 \times 1 + .08 \times 20 = 1.2$$

$$\hat{y}_{woman40} = -.8 + .4 \times 0 + .08 \times 40 = 2.4$$

You have done it before!

Regression

1. Based on your data, you estimate some regression equation
$$y_i = \alpha + \beta_1 x_{i1} + \cdots + \beta_p x_{ip} + \varepsilon_i$$
2. Even if you have some *new unseen data*, you can estimate your expected outcome $\hat{y}$!
3. Example: You estimated a regression equation where y is newspaper reading in days/week:
$$y = -.8 + .4 \times man + .08 \times age$$

4. You could now calculate $\hat{y}$ for a man of 20 years and a woman of 40 years – *even if no such person exists in your dataset*:

$$\hat{y}_{man20} = -.8 + .4 \times 1 + .08 \times 20 = 1.2$$

$$\hat{y}_{woman40} = -.8 + .4 \times 0 + .08 \times 40 = 2.4$$

You have done it before!

Regression

1. Based on your data, you estimate some regression equation
$$y_i = \alpha + \beta_1 x_{i1} + \cdots + \beta_p x_{ip} + \varepsilon_i$$
2. Even if you have some *new unseen data*, you can estimate your expected outcome $\hat{y}$!
3. Example: You estimated a regression equation where y is newspaper reading in days/week:
$$y = -.8 + .4 \times man + .08 \times age$$
4. You could now calculate $\hat{y}$ for a man of 20 years and a woman of 40 years – *even if no such person exists in your dataset*:
$$\hat{y}_{man20} = -.8 + .4 \times 1 + .08 \times 20 = 1.2$$
$$\hat{y}_{woman40} = -.8 + .4 \times 0 + .08 \times 40 = 2.4$$

This is
Supervised Machine Learning!

... but ...

- We will only use *half* (or another fraction) of our data to estimate the model, so that we can use the other half to check if our predictions match the manual coding (“labeled data”, “annotated data” in SML-lingo)
 - e.g., 2000 labeled cases, 1000 for training, 1000 for testing — if successful, run on 100,000 unlabeled cases
- We use many more independent variables (“features”)
- In BoW, IVs are (weighted) word frequencies; for transformers, they are embeddings

... but ...

- We will only use *half* (or another fraction) of our data to estimate the model, so that we can use the other half to check if our predictions match the manual coding (“labeled data”, “annotated data” in SML-lingo)
 - e.g., 2000 labeled cases, 1000 for training, 1000 for testing — if successful, run on 100,000 unlabeled cases
- We use many more independent variables (“features”)
- In BoW, IVs are (weighted) word frequencies; for transformers, they are embeddings

... but ...

- We will only use *half* (or another fraction) of our data to estimate the model, so that we can use the other half to check if our predictions match the manual coding (“labeled data”, “annotated data” in SML-lingo)
 - e.g., 2000 labeled cases, 1000 for training, 1000 for testing — if successful, run on 100,000 unlabeled cases
- We use many more independent variables (“features”)
 - In BoW, IVs are (weighted) word frequencies; for transformers, they are embeddings

... but ...

- We will only use *half* (or another fraction) of our data to estimate the model, so that we can use the other half to check if our predictions match the manual coding (“labeled data”, “annotated data” in SML-lingo)
 - e.g., 2000 labeled cases, 1000 for training, 1000 for testing — if successful, run on 100,000 unlabeled cases
- We use many more independent variables (“features”)
- In BoW, IVs are (weighted) word frequencies; for transformers, they are embeddings

Classification tasks

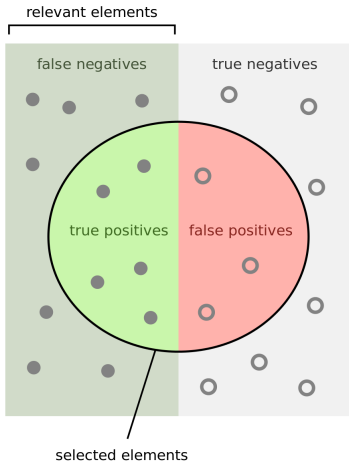
Typical questions include:

- Is this article about topic A, B, C, D, or E?
- Is this review positive or negative?
- Does this text contain frame F?
- Is this satire?
- Is this misinformation?
- Given past behavior, can I predict the next click?

Evaluation

Performance: what do we do and how to evaluate it

Points inside the circle are "annotated positive" by the classifier.
Yellow highlight: annotated points (true positives & false positives).



Some measures

- Accuracy
- Recall
- Precision
- $F1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$

How many selected items are relevant?

$$\text{Precision} = \frac{\text{true positives}}{\text{true positives} + \text{false positives}}$$

How many relevant items are selected?

$$\text{Recall} = \frac{\text{true positives}}{\text{true positives} + \text{false negatives}}$$

Supervised Machine Learning for Text Classification

Supervised Machine Learning for Text Classification

(Traditional) non-SML approaches

Rule-based Example: The world uncertainty index

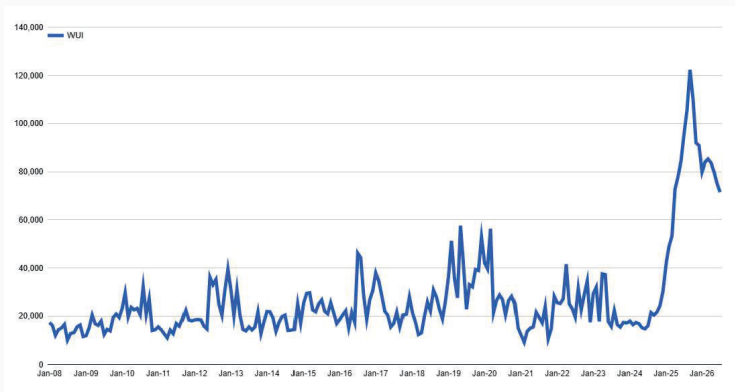
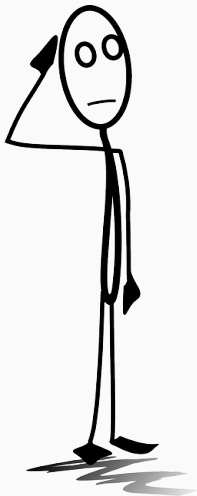


Figure 1: World Uncertainty Index.

The WUI is computed by counting the percent of word “uncertain” (or its variant) in the Economist Intelligence Unit country reports.



Imagine using a dictionary-based (list of keywords, list of regular expressions, or similar) approach to these tasks. How does the design (length, inclusiveness, etc.) of this list influence precision and recall?

Example sentiment dictionary (Minqing Hu and Bing Liu, 2004)

9/17/25, 5:41 PM

positive-words.txt

```

45 | acclamation
46 | accolade
47 | accolades
48 | accommodative
49 | accomodative
50 | accomplish
51 | accomplished
52 | accomplishment
53 | accomplishments
54 | accurate
55 | accurately
56 | achievable
57 | achievement
58 | achievements
59 | achievable
60 | acumen
61 | adaptable
62 | adaptive

```

Dictionary-based approaches for text classification

good for

- distinct, manifest things
(names of organizations,
pronouns, swearwords (?),
...)
- little room for interpreta-
tion/misunderstandings etc.
- “must-be-explainable-to-a-
five-year-old”

bad for

- latent constructs and
concepts
- implicit things

Hence, *not* state-of-the-art for

- topics
- frames
- sentiment

Dictionary-based approaches for text classification

good for

- distinct, manifest things
(names of organizations,
pronouns, swearwords (?),
...)
- little room for interpreta-
tion/misunderstandings etc.
- “must-be-explainable-to-a-
five-year-old”

bad for

- latent constructs and
concepts
- implicit things

Hence, *not* state-of-the-art for

- topics
- frames
- sentiment

Dictionary-based approaches for text classification

good for

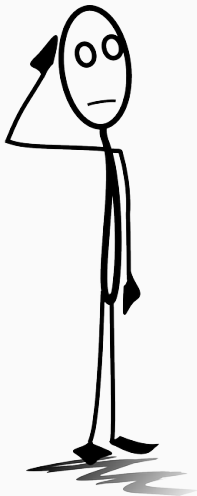
- distinct, manifest things (names of organizations, pronouns, swearwords (?), ...)
- little room for interpretation/misunderstandings etc.
- “must-be-explainable-to-a-five-year-old”

bad for

- latent constructs and concepts
- implicit things

Hence, *not* state-of-the-art for

- topics
- frames
- sentiment



*Can we construct a dictionary that,
irrespective of the context, gives us a
meaningful estimate of sentiment?*

Boukes et al., 2020: Sentiment analysis of economic news

Table A1. Correlations between sentiment scores using different methods for headlines (above) and full texts (below).

	Headline							
	Manual coding	Recession	D & B	LIWC	SentiStrength	Pattern	Polyglot	DANEW
Manual coding	1.00 ***							
Recession	-	-						
Damstra and Boukes (2018)	0.16 ***	-	1.00 ***					
LIWC	0.30 ***	-	0.16 ***	1.00 ***				
SentiStrength	0.24 ***	-	0.08 **	0.26 ***	1.00 ***			
Pattern	0.22 ***	-	0.00	0.30 ***	0.22 ***	1.00 ***		
Polyglot	0.30 ***	-	0.19 ***	0.32 ***	0.37 ***	0.26 ***	1.00 ***	
DANEW	0.24 ***	-	0.04	0.43 ***	0.33 ***	0.23 ***	0.32 ***	1.00 ***

	Full text							
	Manual coding	Recession	D & B	LIWC	SentiStrength	Pattern	Polyglot	DANEW
Manual coding	1.00 ***							
Recession	-0.06 *	1.00 ***						
Damstra and Boukes (2018)	0.27 ***	-0.16 ***	1.00 ***					
LIWC	0.39 ***	0.02	0.27 ***	1.00 ***				
SentiStrength	0.17 ***	-0.01	0.10 ***	0.18 ***	1.00 ***			
Pattern	0.13 ***	-0.02	0.04	0.28 ***	0.12 ***	1.00 ***		
Polyglot	0.26 ***	0.05	0.17 ***	0.41 ***	0.21 ***	0.30 ***	1.00 ***	
DANEW	0.15 ***	0.06 *	0.05	0.36 ***	0.18 ***	0.29 ***	0.37 ***	1.00 ***

The word "recession" did not occur in headlines of our sample, as such, no correlation coefficient is available for the recession classifier; *** $p < .001$, ** $p < .010$, * $p < .05$.

Boukes et al., 2020: Sentiment analysis of economic news

- Dictionaries have low agreement with each other, and also with human coders
- Even their own dictionary didn't agree
- **This is not because these dictionaries are particularly bad!** Main point: For such a complex and context-dependent task, a dictionary is just not the right tool.

van Atteveldt et al., 2021: Extending Boukes et al., 2020 with SML

1. **Manual coding** (undergraduate students)
“yields the best results”
2. Crowd coding
“a good second place”
3. Machine learning
worse than manual & crowd coding, but significantly better than chance
• deep learning (CNN): $\alpha = 0.39$
• classical ML (NB): $\alpha = 0.40$
4. Dictionaries
worse than ML, much worse than manual annotation —
approximates chance agreement

van Atteveldt et al., 2021: Extending Boukes et al., 2020 with SML

1. **Manual coding** (undergraduate students)
“yields the best results”
2. **Crowd coding**
“a good second place”
3. **Machine learning**
worse than manual & crowd coding, but significantly better than chance
 • Deep learning (CNN): $\kappa = 0.39$
 • Classical ML (NB): $\kappa = 0.40$
4. **Dictionaries**
worse than ML, much worse than manual annotation — approximates chance agreement

van Atteveldt et al., 2021: Extending Boukes et al., 2020 with SML

1. **Manual coding** (undergraduate students)
“yields the best results”
2. **Crowd coding**
“a good second place”
3. **Machine learning**
worse than manual & crowd coding, but significantly better than chance
 - deep learning (CNN): $\alpha = 0.50$
 - classical ML (NB): $\alpha = 0.40$
4. **Dictionaries**
worse than ML, much worse than manual annotation — approximates chance agreement

van Atteveldt et al., 2021: Extending Boukes et al., 2020 with SML

1. **Manual coding** (undergraduate students)
“yields the best results”
2. **Crowd coding**
“a good second place”
3. **Machine learning**
worse than manual & crowd coding, but significantly better than chance
 - deep learning (CNN): $\alpha = 0.50$
 - classical ML (NB): $\alpha = 0.40$
4. **Dictionaries**
worse than ML, much worse than manual annotation — approximates chance agreement

van Atteveldt et al., 2021: Extending Boukes et al., 2020 with SML

1. **Manual coding** (undergraduate students)
“yields the best results”
2. **Crowd coding**
“a good second place”
3. **Machine learning**
worse than manual & crowd coding, but significantly better than chance
 - deep learning (CNN): $\alpha = 0.50$
 - classical ML (NB): $\alpha = 0.40$
4. **Dictionaries**
worse than ML, much worse than manual annotation — approximates chance agreement

van Atteveldt et al., 2021: Extending Boukes et al., 2020 with SML

1. **Manual coding** (undergraduate students)
“yields the best results”
2. **Crowd coding**
“a good second place”
3. **Machine learning**
worse than manual & crowd coding, but significantly better than chance
 - deep learning (CNN): $\alpha = 0.50$
 - classical ML (NB): $\alpha = 0.40$
4. **Dictionaries**
worse than ML, much worse than manual annotation —
approximates chance agreement

Vermeer et al., 2019: Satisfaction with brands

Category	Technique	Accuracy	Precision	Recall
Satisfaction (N = 854)				
Sentiment analysis	LIWC	0.05	0.06	0.04
	P	0.04	0.04	0.04
	SN	0.07	0.07	0.08
Dictionary-based	D	0.15	0.30	0.10
Machine learning	BNB	0.38	0.44	0.34
	MNB	0.32	0.67	0.21
	LR	0.51	0.38	0.76
	SGD	0.49	0.38	0.69
	SVM	0.52	0.41	0.63
	PA	0.50	0.40	0.68
Neutral (N = 760)				
Sentiment analysis	LIWC	0.13	0.16	0.10
	P	0.13	0.13	0.14
	SN	0.19	0.16	0.22
Dictionary-based	D	0.14	0.35	0.09
Machine learning	BNB	0.28	0.25	0.32
	MNB	0.15	0.34	0.10
	LR	0.37	0.25	0.74
	SGD	0.33	0.23	0.60
	SVM	0.36	0.24	0.69
	PA	0.34	0.24	0.60
Dissatisfaction (N = 267)				
Sentiment analysis	LIWC	0.20	0.15	0.29
	P	0.19	0.12	0.40
	SN	0.22	0.14	0.54
Dictionary-based	D	0.09	0.41	0.05
Machine learning	BNB	0.26	0.20	0.40
	MNB	0.25	0.48	0.16
	LR	0.35	0.23	0.77
	SGD	0.39	0.32	0.48
	SVM	0.04	0.02	1.00
	PA	0.35	0.23	0.71

Note. LIWC Linguistic Inquiry and Word Count; P Pattern; SN, Sentiment Net; D Dictionary-based; BN Bernoulli Naïve Bayes; MNB Multinomial Naïve Bayes; LR Logistic Regression; SGD Stochastic Gradient Descent; SVM Support Vector Machine; and PA Passive Aggressive. Performance scores ≥ 0.60 have been highlighted. Results merely derived from the test set.

SML is no panacea, but it introduces a promising approach to analyzing large quantities of texts. Don't believe off-the-shelf packages that claim to do the work for you. (For small datasets, just do it by hand.)

Supervised Machine Learning for Text Classification

Traditional Machine Learning

Different classification algorithms

Examples: Naïve Bayes, Logistic Regression, Support Vector Machines etc.

- It is an empirical question which one works best
- You typically try several ones and select the best
- (remember: we have a test dataset that we did *not* use to train the model, so that we can assess how well it predicts the test labels based on the test features)
- To avoid p -hacking-like scenarios (which we call “overfitting”), there are techniques available (e.g., cross-validation, which we address tomorrow)

Different classification algorithms

Examples: Naïve Bayes, Logistic Regression, Support Vector Machines etc.

- It is an empirical question which one works best
- You typically try several ones and select the best
- (remember: we have a test dataset that we did *not* use to train the model, so that we can assess how well it predicts the test labels based on the test features)
- To avoid *p*-hacking-like scenarios (which we call "overfitting"), there are techniques available (e.g., cross-validation, which we address tomorrow)

Different classification algorithms

Examples: Naïve Bayes, Logistic Regression, Support Vector Machines etc.

- It is an empirical question which one works best
- You typically try several ones and select the best
- (remember: we have a test dataset that we did *not* use to train the model, so that we can assess how well it predicts the test labels based on the test features)
- To avoid *p*-hacking-like scenarios (which we call "overfitting"), there are techniques available (e.g., cross-validation, which we address tomorrow)

Different classification algorithms

Examples: Naïve Bayes, Logistic Regression, Support Vector Machines etc.

- It is an empirical question which one works best
- You typically try several ones and select the best
- (remember: we have a test dataset that we did *not* use to train the model, so that we can assess how well it predicts the test labels based on the test features)
- To avoid p -hacking-like scenarios (which we call “overfitting”), there are techniques available (e.g., cross-validation, which we address tomorrow)

Naïve Bayes

Unsophisticated, usually a baseline

Bayes' theorem

$$P(A \mid B) = \frac{P(B \mid A) \times P(A)}{P(B)}$$

A = Text is about sports

B = Text contains 'very', 'close', 'game'. Furthermore, we simply multiply the probabilities for the features:

$$P(B) = P(\text{very close game}) = P(\text{very}) \times P(\text{close}) \times P(\text{game})$$

We can fill in all values by counting how many articles are about sports, and how often the words occur in these texts.

Naïve Bayes

Unsophisticated, usually a baseline

Bayes' theorem

$$P(A \mid B) = \frac{P(B \mid A) \times P(A)}{P(B)}$$

A = Text is about sports

B = Text contains 'very', 'close', 'game'. Furthermore, we simply multiply the probabilities for the features:

$$P(B) = P(\text{very close game}) = P(\text{very}) \times P(\text{close}) \times P(\text{game})$$

We can fill in all values by counting how many articles are about sports, and how often the words occur in these texts.

Naïve Bayes

Unsophisticated, usually a baseline

Bayes' theorem

$$P(A \mid B) = \frac{P(B \mid A) \times P(A)}{P(B)}$$

A = Text is about sports

B = Text contains 'very', 'close', 'game'. Furthermore, we simply multiply the probabilities for the features:

$$P(B) = P(\text{very close game}) = P(\text{very}) \times P(\text{close}) \times P(\text{game})$$

We can fill in all values by counting how many articles are about sports, and how often the words occur in these texts.

Naïve Bayes

- For computers, calculating this is *really easy and fast*
- Weird assumption: features are independent
- Features predict classes linearly

Supervised Machine Learning for Text Classification

The implementation

An implementation

Let's say we have two lists, with movie reviews and their rating:

```
1 reviews_train = ["This is a great movie", "Bad movie", ... ...]
2 labels_train = [1,-1, ...]
```

And a second dataset with an identical structure:

```
1 reviews_test = ["Not that good","Nice film", ... ...]
2 labels_test = [-1,1, .....]
```

Both are drawn from the same population; it is pure chance whether a specific review is on the one list or the other.

Based on an example from <http://blog.dataquest.io/blog/naive-bayes-movies/>

Training a Naïve Bayes Classifier

```

1 from sklearn.naive_bayes import MultinomialNB
2 from sklearn.feature_extraction.text import CountVectorizer
3 from sklearn import metrics
4
5 vectorizer = CountVectorizer(stop_words='english')
6 features_train = vectorizer.fit_transform(reviews_train)
7 features_test = vectorizer.transform(reviews_test)
8
9 # Fit a naive bayes model to the training data.
10 nb = MultinomialNB()
11 nb.fit(features_train, labels_train)
12
13 # Now we can use the model to predict classifications for our test
   features.
14 predictions = nb.predict(features_test)
15
16 print(f"Precision:\t{metrics.precision_score(labels_test, predictions,
   pos_label=1, labels = [-1,1])}")
17 print(f"Recall:\t{metrics.recall_score(labels_test, predictions,
   pos_label=1, labels = [-1,1])}")

```

And it works!

Using 50,000 IMDB movies that are classified as either negative or positive,

- We created a list with 25,000 training tuples and another one with 25,000 test tuples and
- trained a classifier
- with precision and recall values $> .80$

Dataset obtained from <http://ai.stanford.edu/~amaas/data/sentiment>, Maas, A.L., Daly, R.E., Pham, P.T., Huang, D., Ng, A.Y., & Potts, C. (2011). Learning word vectors for sentiment analysis. *49th Annual Meeting of the Association for Computational Linguistics (ACL 2011)*

Playing around with new data

```
1 newdata=vectorizer.transform(["What a crappy movie! It sucks!", "This is  
    awesome. I liked this movie a lot, fantastic actors","I would not  
    recomment it to anyone.", "Enjoyed it a lot"])  
2 predictions = nb.predict(newdata)  
3 print(predictions)
```

This returns, as you would expect and hope:

```
1 [-1  1 -1  1]
```

But we can do even better

We can use different vectorizers and different classifiers.

Different classifiers

Typical options in a nutshell:

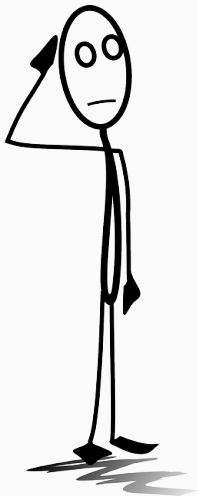
- Naïve Bayes
- Logistic Regression
- Support Vector Machine (SVM/SVC)
- Random forests

Different vectorizers

- CountVectorizer
- TfidfVectorizer

Preprocessing:

- Stopword removal
- Pruning
- etc.



Any questions?

ML versus dictionaries

What *is* our fitted classifier again?

Essentially, just a formula

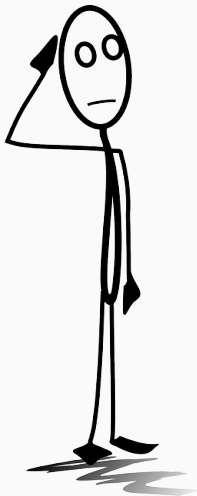
$$p = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n)}}$$

where β_0 is an intercept¹, β_1 a coefficient for the frequency (or tf·idf score) of some word, β_2 a coefficient some other word.

If our fitted *vectorizer* contains 5,000 words, we thus have 5,001 coefficients.

(for logistic regression in this case, but same argument applies to other classifiers as well)

¹Machine Learning people sometimes call the intercept “bias” (yes, I know, that’s confusing)



But isn't that then essentially very much like a dictionary, except that the words have different weights?

In some sense, yes.

- But we don't pretend that we can construct the dictionary *a priori*.
- It's specifically tailored to our use case.
- The weights are *really* essential here.

We *could* print all coefficient-word pairs, but probably it's enough to just look at those with the largest absolute value:

In some sense, yes.

- But we don't pretend that we can construct the dictionary *a priori*.
- It's specifically tailored to our use case.
- The weights are *really* essential here.

We *could* print all coefficient-word pairs, but probably it's enough to just look at those with the largest absolute value:

ML versus dictionaries

The explainability of ML

35

ELI5

- Inspecting *all* coefficients of a ML model usually doesn't make much sense
- But that does not mean that we cannot understand how the model makes its predictions
- We can look at the most important coefficients
- We can look which words in a given text contributed most to its classification

Have we solved all problems of dictionaries?

No.

For instance, the negation and/or intensifier problem.

Possible approaches

- n -grams as features
- preprocessing (?)
- deep learning
- ...

⇒ But ultimately, it's just an empirical question how big the problem is!

Have we solved all problems of dictionaries?

No.

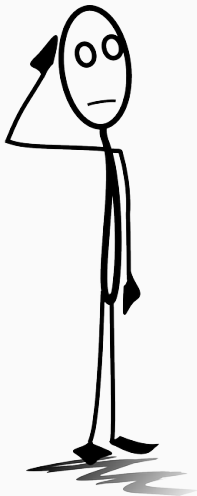
For instance, the negation and/or intensifier problem.

Possible approaches

- n -grams as features
- preprocessing (?)
- deep learning
- ...

⇒ But ultimately, it's just an empirical question how big the problem is!

Exercise



Any questions?

This afternoon: SML with transformers.

References i

References



Boukes, M., van de Velde, B., Araujo, T., & Vliegthart, R. (2020). **What's the Tone? Easy Doesn't Do It: Analyzing Performance and Agreement Between Off-the-Shelf Sentiment Analysis Tools.** *Communication Methods and Measures*, 14(2), 83–104.
<https://doi.org/10.1080/19312458.2019.1671966>



Boumans, J. W., & Trilling, D. (2016). Taking stock of the toolkit: An overview of relevant automated content analysis approaches and techniques for digital journalism scholars. *Digital Journalism*, 4(1), 8–23. <https://doi.org/10.1080/21670811.2015.1096598>

References ii



van Atteveltdt, W., van der Velden, M. A., & Boukes, M. (2021).
**The Validity of Sentiment Analysis:Comparing
 Manual Annotation, Crowd-Coding, Dictionary
 Approaches, and Machine Learning Algorithms.**
Communication Methods and Measures, 00(00), 1–20.
<https://doi.org/10.1080/19312458.2020.1869198>



Vermeer, S., Araujo, T., Bernritter, S. F., & van Noort, G. (2019).
**Seeing the wood for the trees: How machine learning
 can help firms in identifying relevant electronic
 word-of-mouth in social media.** *International Journal of
 Research in Marketing*, 36(3), 492–508.
<https://doi.org/10.1016/j.ijresmar.2019.01.010>